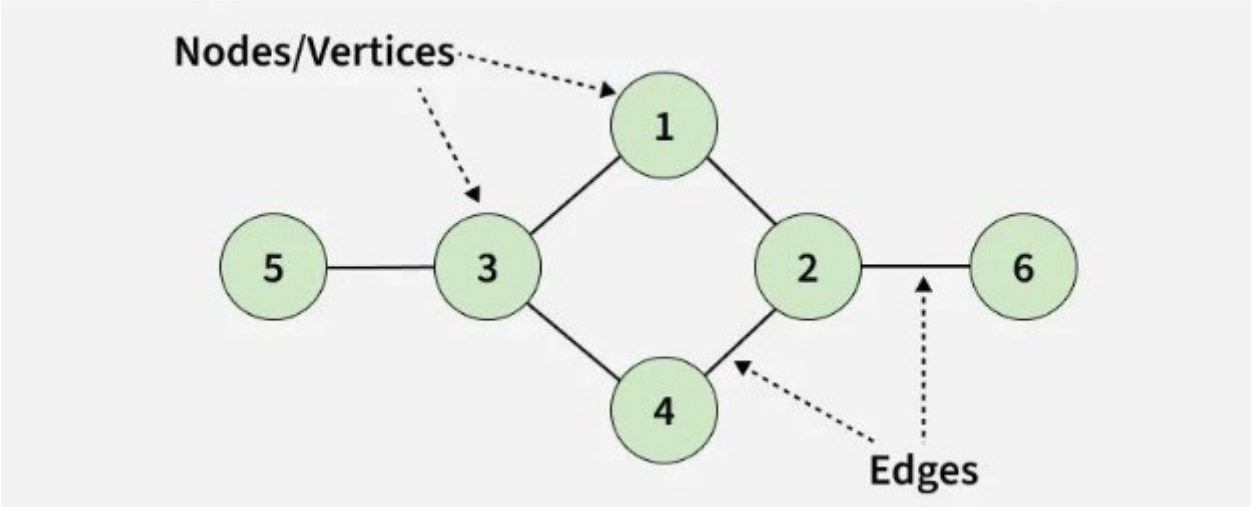
Lecture 12

Graphs



Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group
- Project 1 Final Report due **Monday** @ 11:59pm
  - Presentations on Monday (7 to 10 minutes)
  - See example project on Github
  - You received feedback via email
- Exam 1 rewrites due next Thursday at the beginning of lecture via email
  - Can earn back up to 50% on any question (e.g. 3/10 → 5/10)
  - Provide a very thorough answer and write up in a markdown file

Last time

Introduce new data structures

- Linked lists
- Doubly linked lists

Lab: introduction to graphs

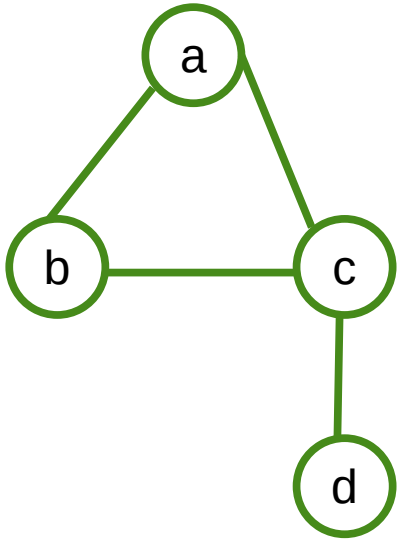
Today

- Graphs & Trees
- Traversals
- Breadth-First Search (BFS)

Graphs

Mathematical representation of a network

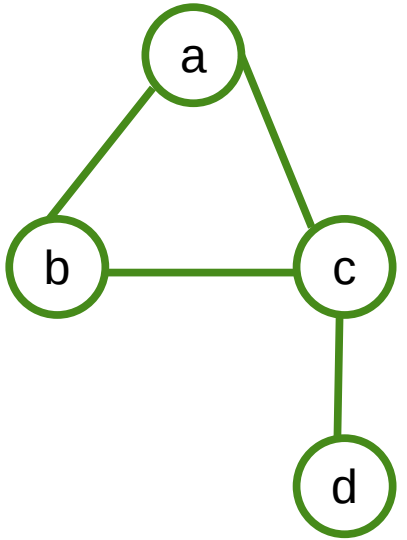
- Set of vertices V (also called nodes)
- Set of edges E (pairs of vertices)



Graphs

Mathematical representation of a network

- Set of vertices `V` (also called nodes)
- Set of edges `E` (pairs of vertices)



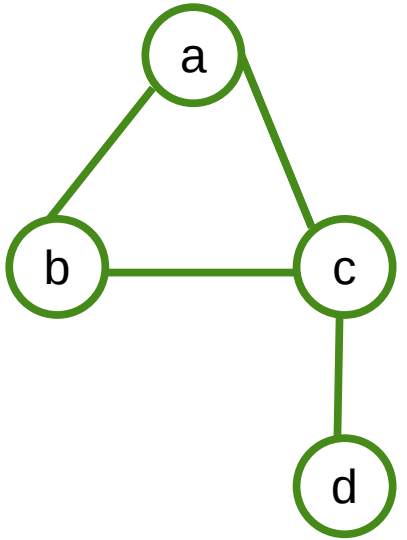
Graph `G = (V,E)`

- `G` has `n = |V|` vertices
- `G` has `m = |E|` edges

Paths

A **path** is a sequence of vertices
$P = v_1, v_2, \ldots, v_k$ such that $v_i v_{i+1}$ is in E

A **simple path** is a path with distinct vertices



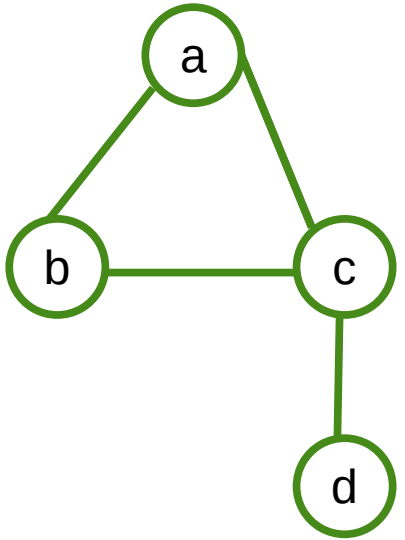
Paths

A **path** is a sequence of vertices
P = $v_1, v_2, \ldots, v_k$ such that $v_i v_{i+1}$ is in E

A **simple path** is a path with distinct vertices

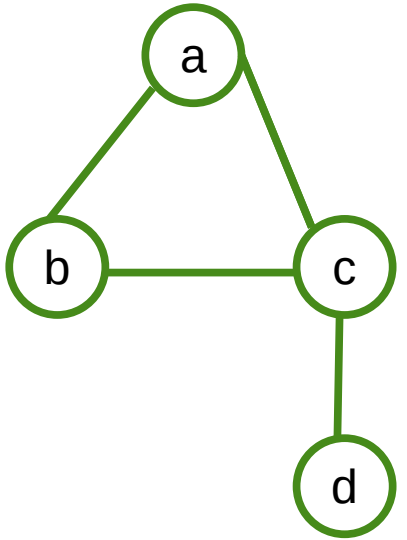
A **cycle** is a path C = $v_1, v_2, \ldots, v_{k-1}, v_k$ where

- $v_1 = v_k$
- $v_1, v_2, \ldots, v_{k-1}$ is a simple path
- All edges are distinct



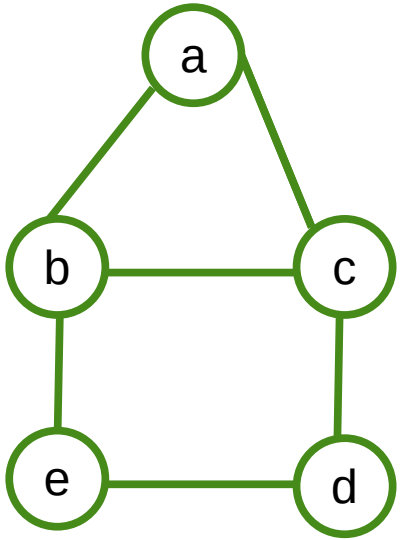
Distance

- The **length** of a path is the number of edges it contains
- A path from vertex u to vertex v is called a **uv path**
- The **distance** from u to v is the length of the shortest path uv path



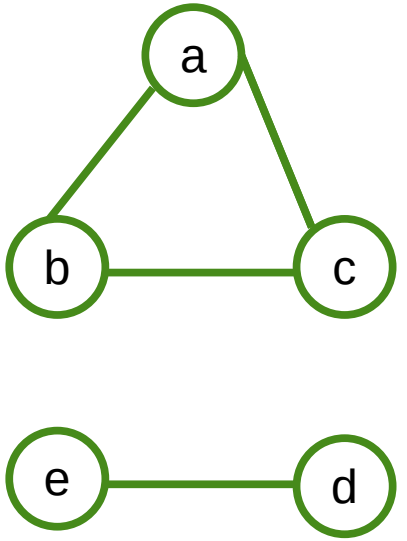
Connectedness

- A **connected graph** has paths between every pair of vertices



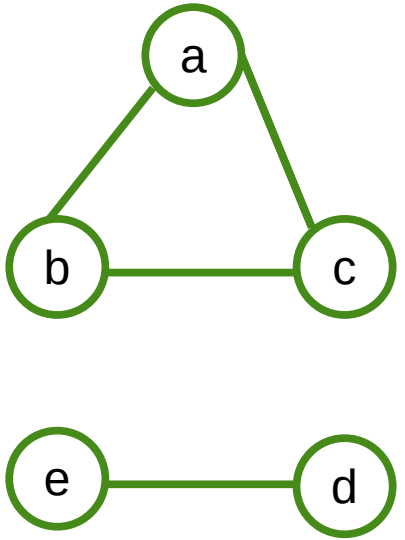
Connectedness

- A **connected graph** has paths between every pair of vertices



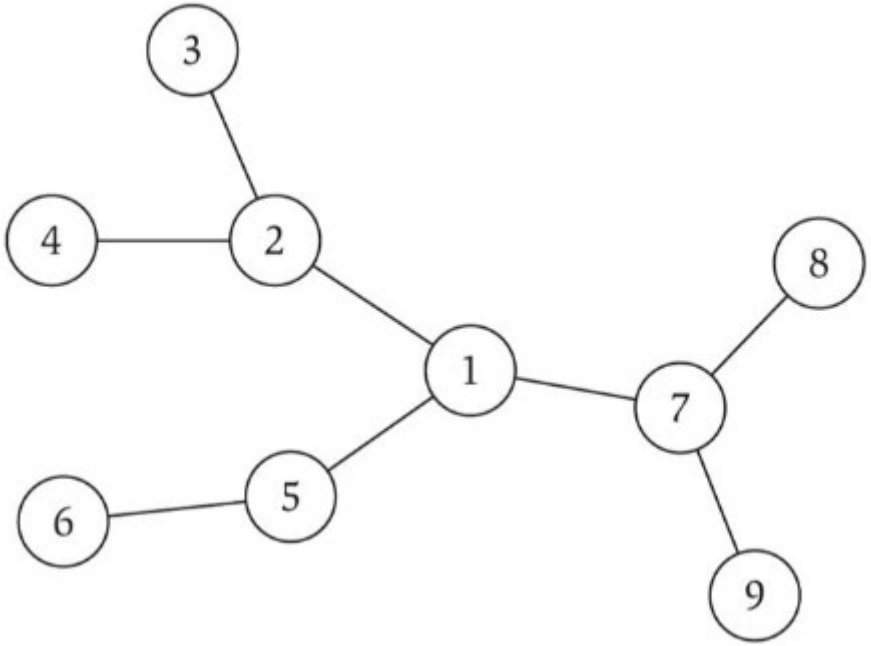
Connectedness

- A **connected graph** has paths between every pair of vertices
- A **connected component** is a maximal subset of vertices with paths between every pair
- **Maximal** means that if a new vertex is added there will no longer be a path between every pair



Trees

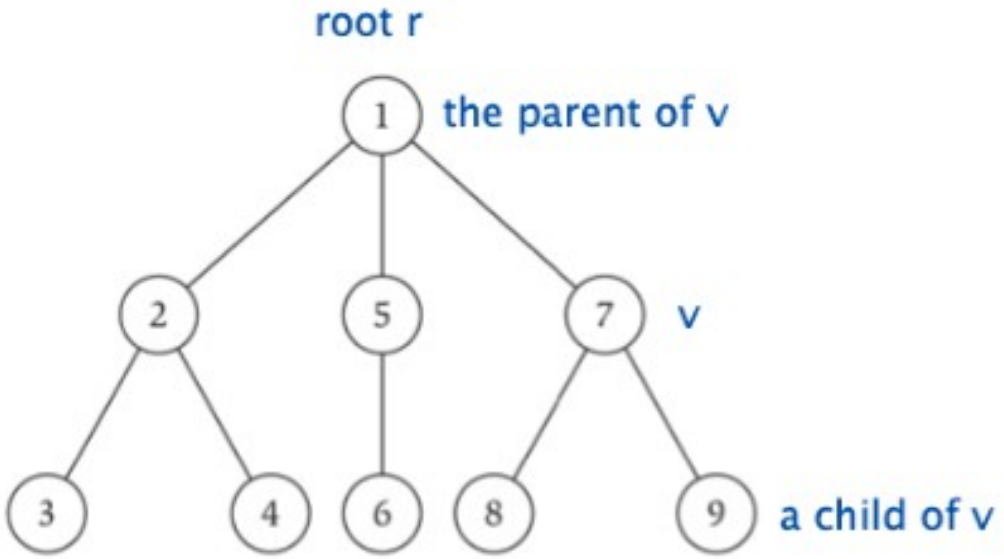
- A **tree** is a connected graph containing no cycles



Trees

- A **tree** is a connected graph containing no cycles

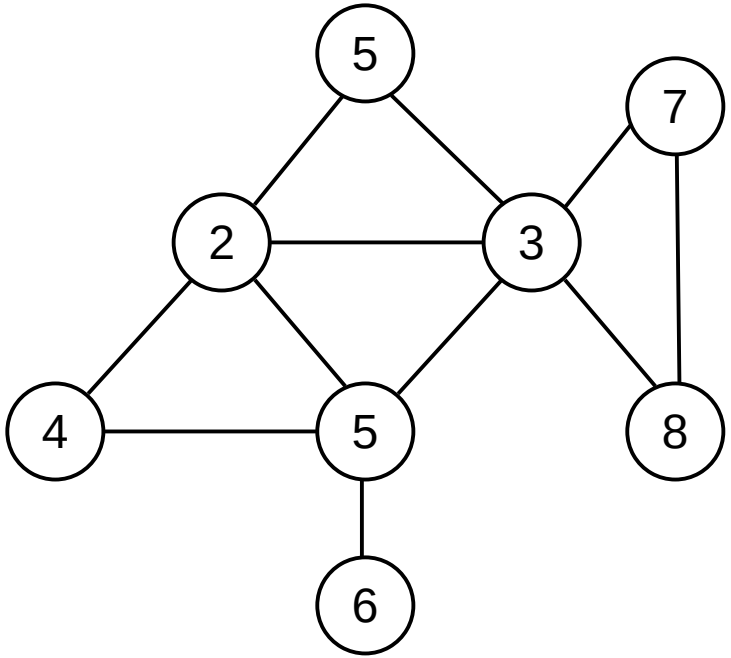
- A **rooted tree** is a tree encoding a parent-child relationship
  - Pick a root and orient all edges away from the root
  - Parent of v means predecessor on the path from r to v



Traversal

Imagine we're plopped down onto a vertex

- How/what can we learn about this graph?
- Is it connected?
- If not, how big is the largest connected component?
- What is the distance from 2 to 8?

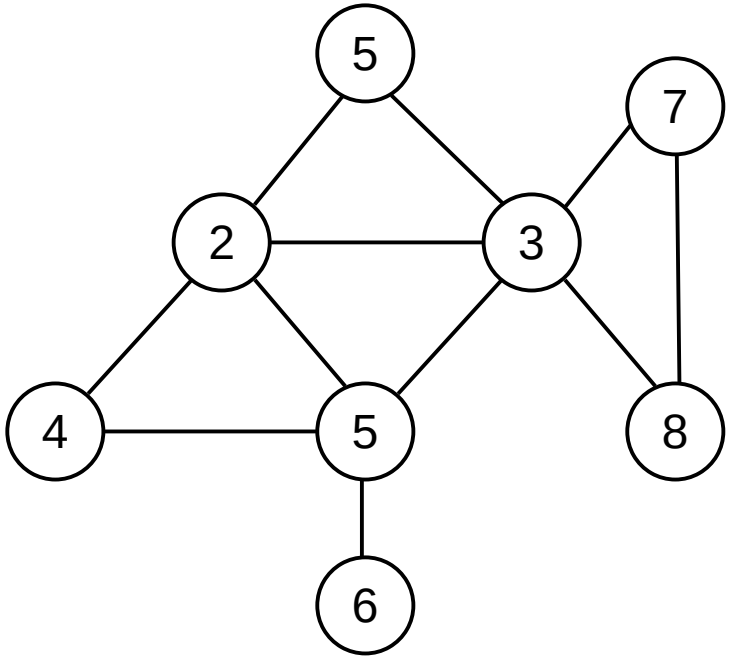


Traversal

Imagine we're plopped down onto a vertex

- How/what can we learn about this graph?
- Is it connected?
- If not, how big is the largest connected component?
- What is the distance from 2 to 8?

Goal: answer these algorithmically



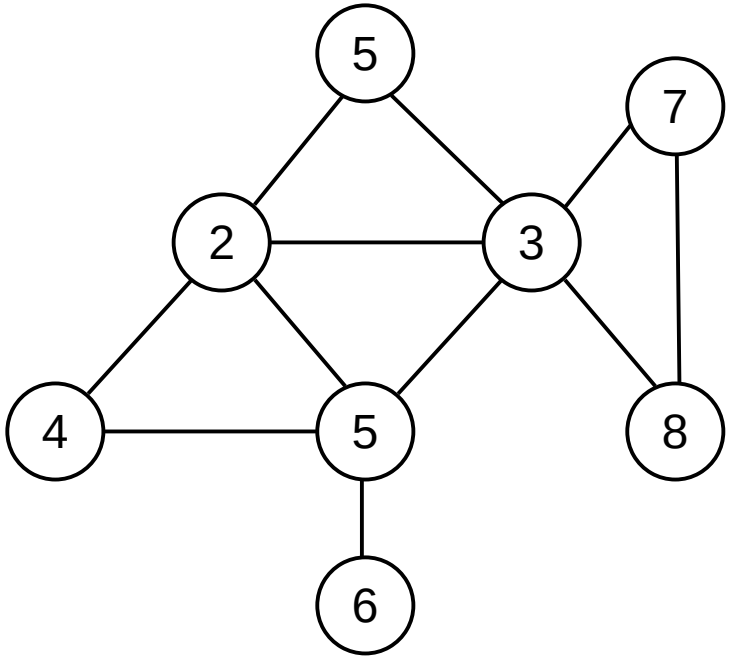
Traversal Strategies

Breadth-First Search (BFS)

- Explore locally

Depth-First Search (DFS)

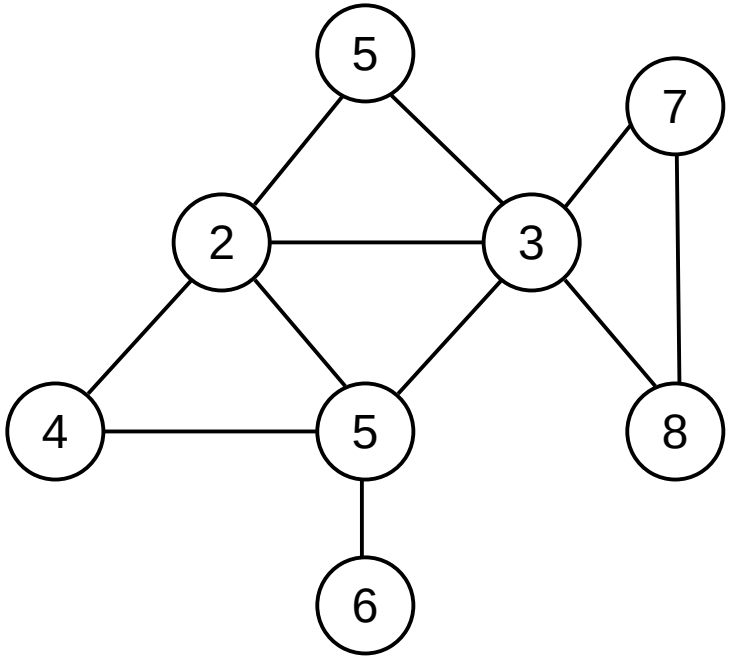
- Deep dive and backtrack



Breadth-First Search

Expand outward from starting vertex by distance

- "Expanding wave"

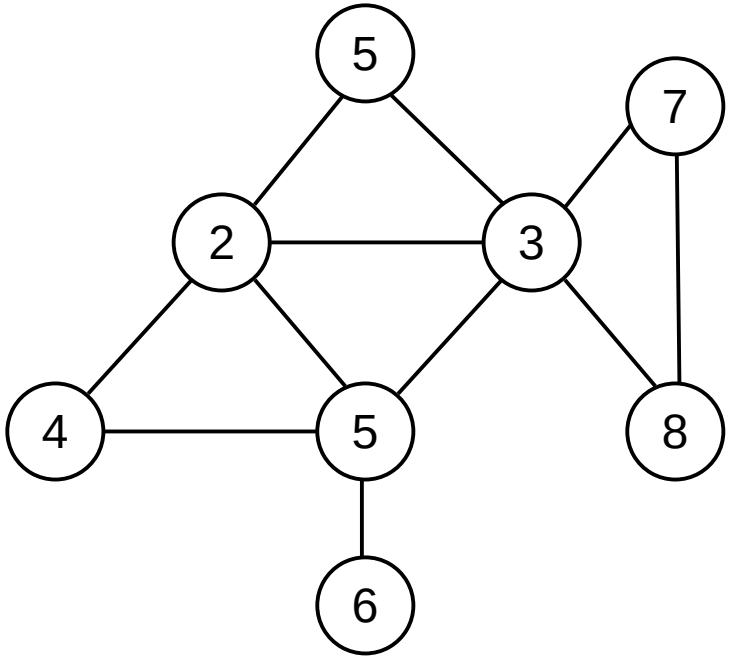


Breadth-First Search

Expand outward from starting vertex by distance

- "Expanding wave"

Let's start at vertex 4



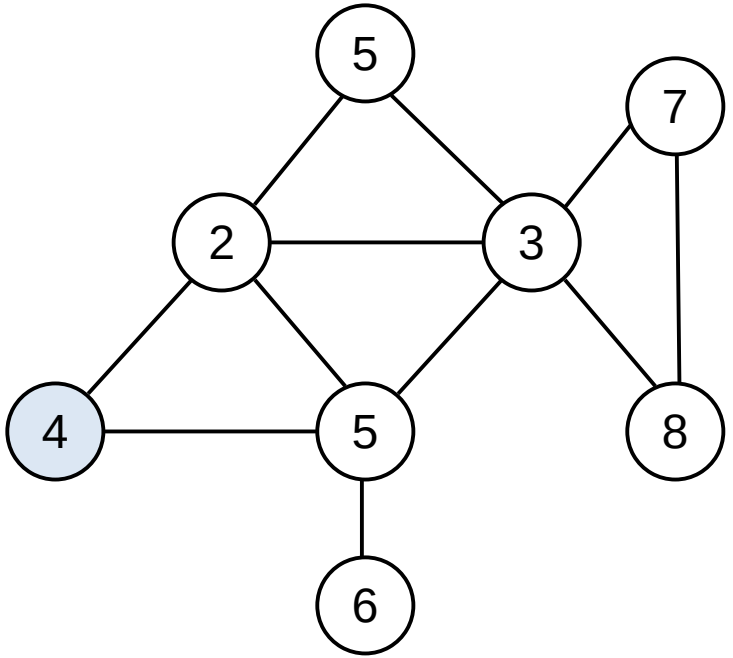
Breadth-First Search

Expand outward from starting vertex by distance

- "Expanding wave"

Let's start at vertex 4

- Distance 0 from 4



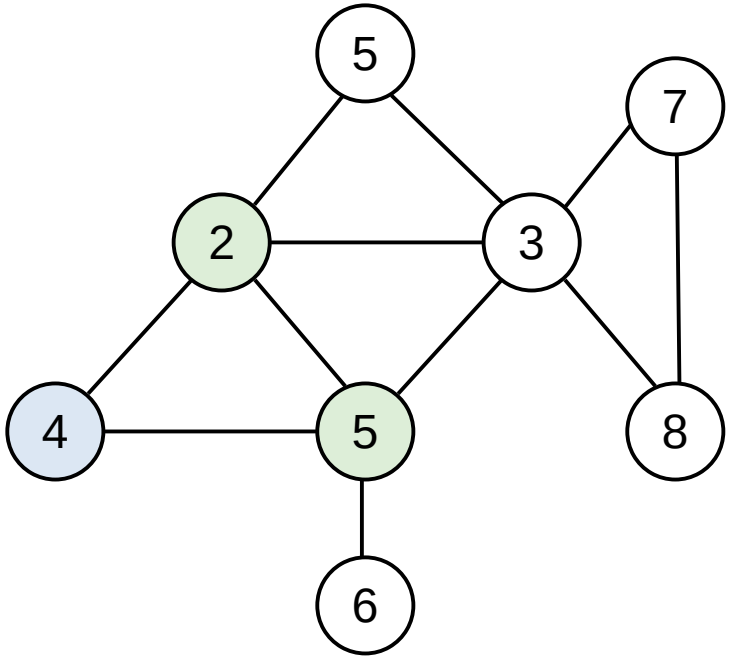
Breadth-First Search

Expand outward from starting vertex by distance

- "Expanding wave"

Let's start at vertex 4

- Distance 0 from 4
- Distance 1 from 4



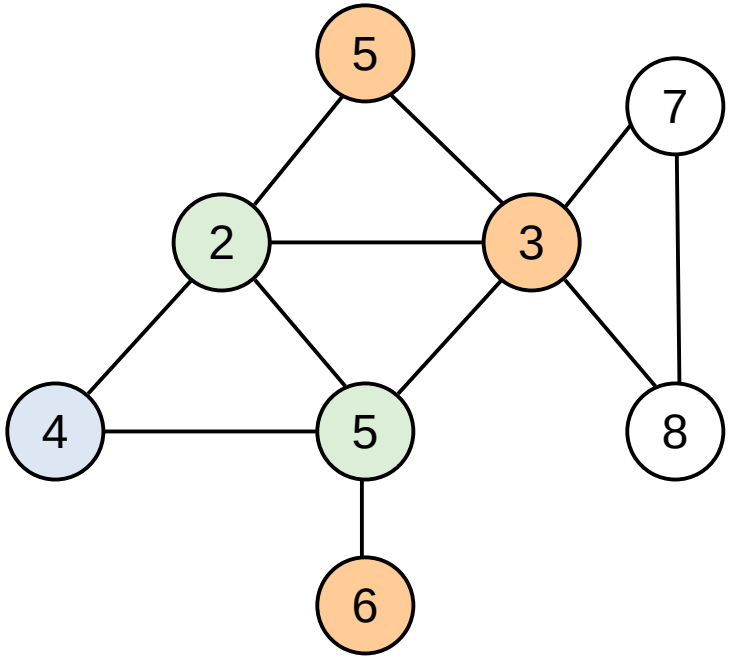
Breadth-First Search

Expand outward from starting vertex by distance

- "Expanding wave"

Let's start at vertex 4

- Distance 0 from 4
- Distance 1 from 4
- Distance 2 from 4



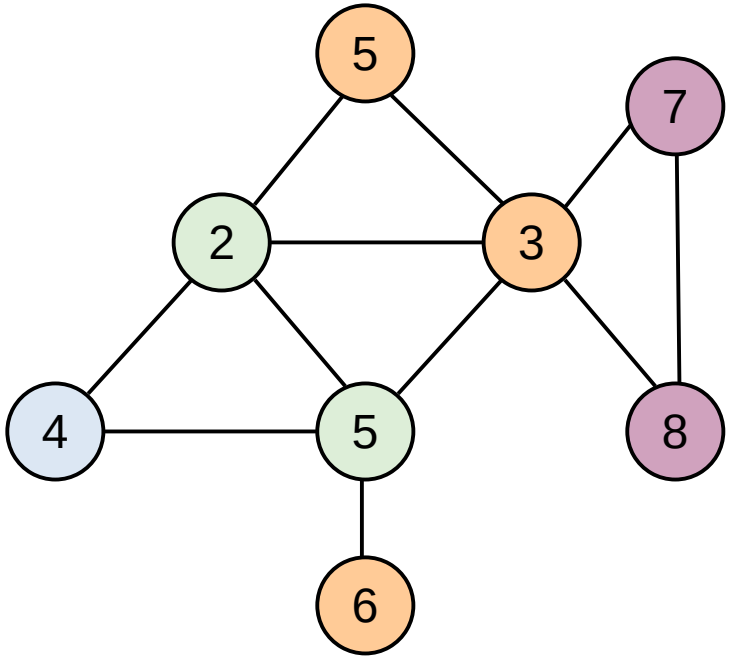
Breadth-First Search

Expand outward from starting vertex by distance

- "Expanding wave"

Let's start at vertex 4

- Distance 0 from 4
- Distance 1 from 4
- Distance 2 from 4
- Distance 3 from 4



Breadth-First Search

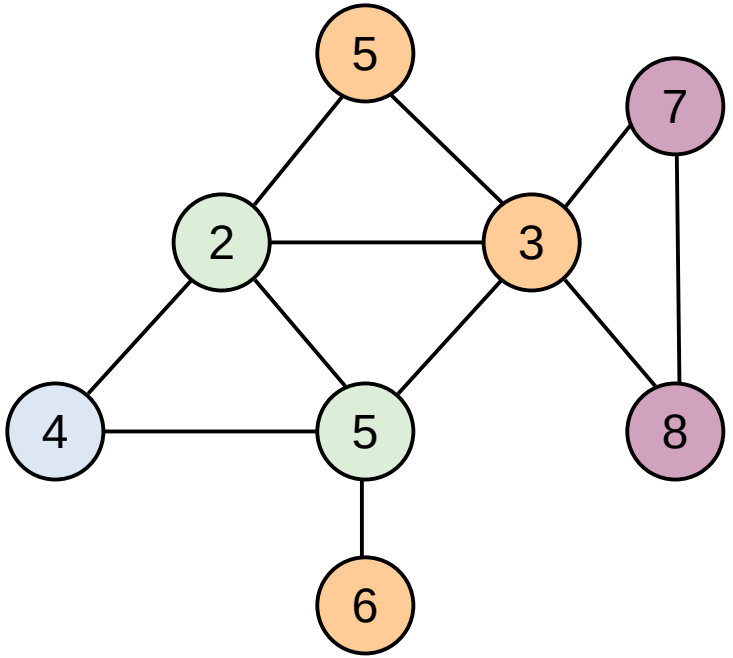
Expand outward from starting vertex by distance

- "Expanding wave"

Let's start at vertex 4

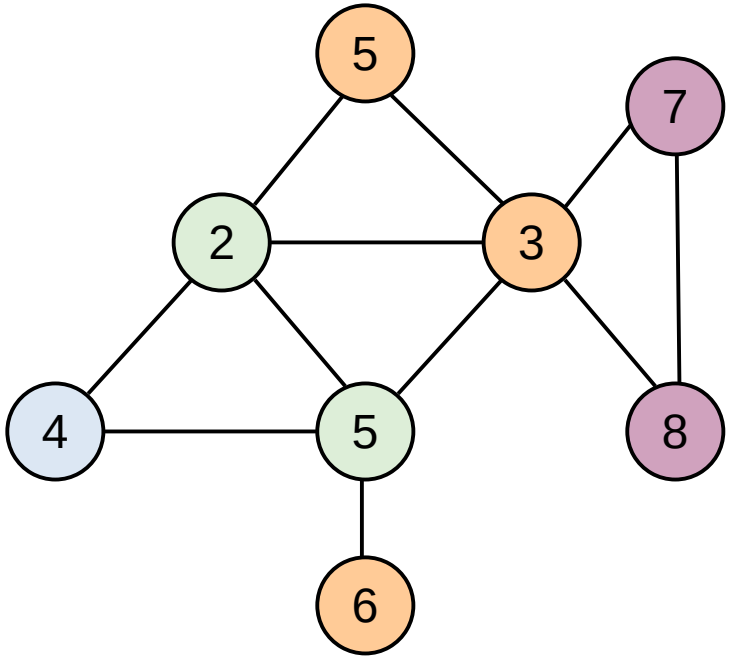
- Distance 0 from 4
- Distance 1 from 4
- Distance 2 from 4
- Distance 3 from 4

All vertices reachable from 4 will be explored eventually



Breadth-First Search

- Let L_0 denote the set containing the root
- Define L_{i+1} from L_i as the neighbors of vertices in L_i that have not yet been visited
- Continue until no new vertices are found

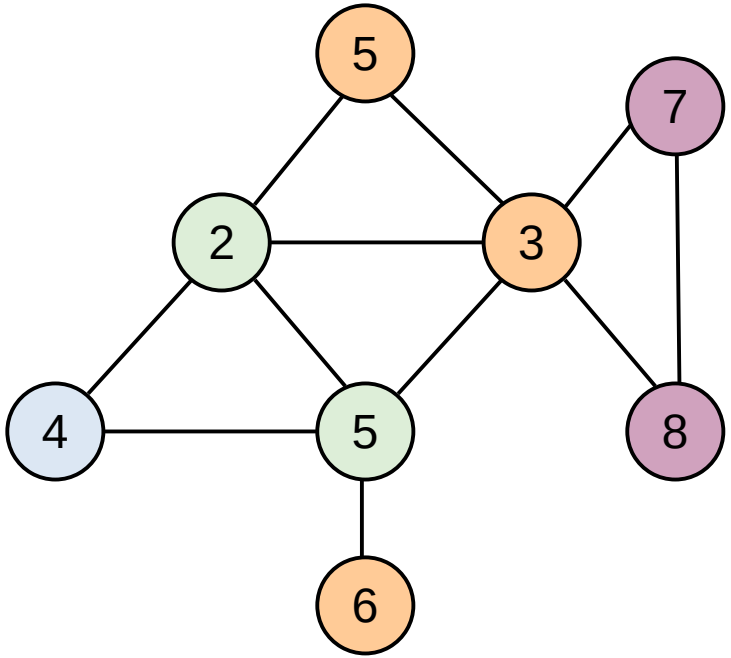


Breadth-First Search

- Let L_0 denote the set containing the root
- Define L_{i+1} from L_i as the neighbors of vertices in L_i that have not yet been visited
- Continue until no new vertices are found

Complexity: $O(n+m)$

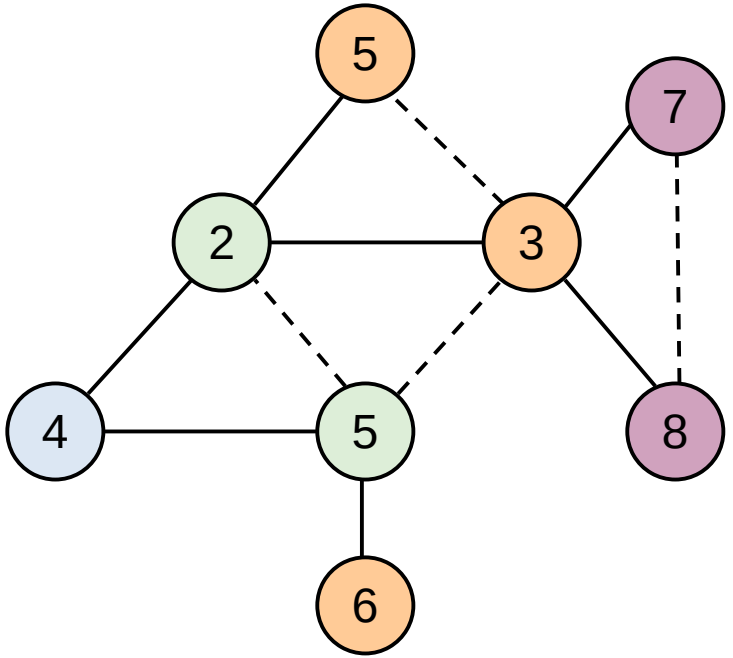
- # vertices + # edges
- Efficiently implement using a queue



BFS Tree

We can use BFS to make a tree

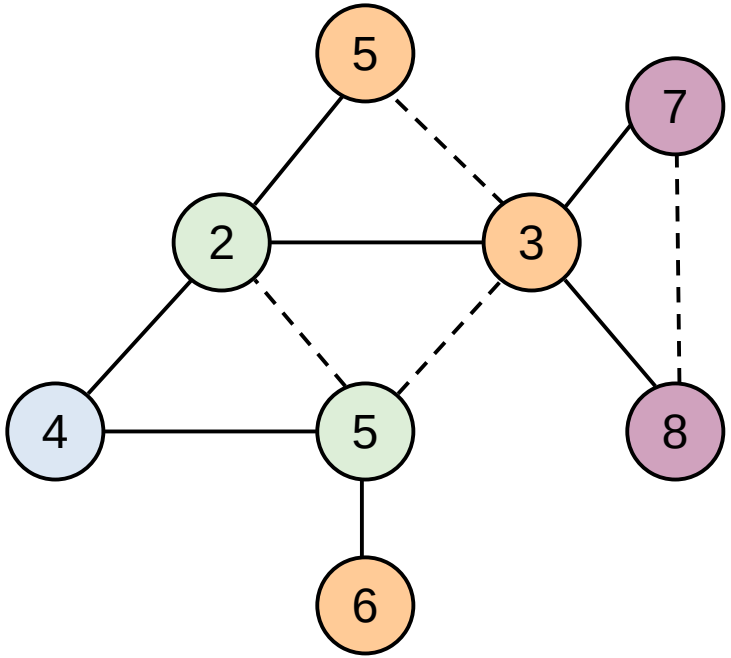
- Keep edge uv if v was first visited as a neighbor of u
- Why does BFS make a tree?



BFS Tree

We can use BFS to make a tree

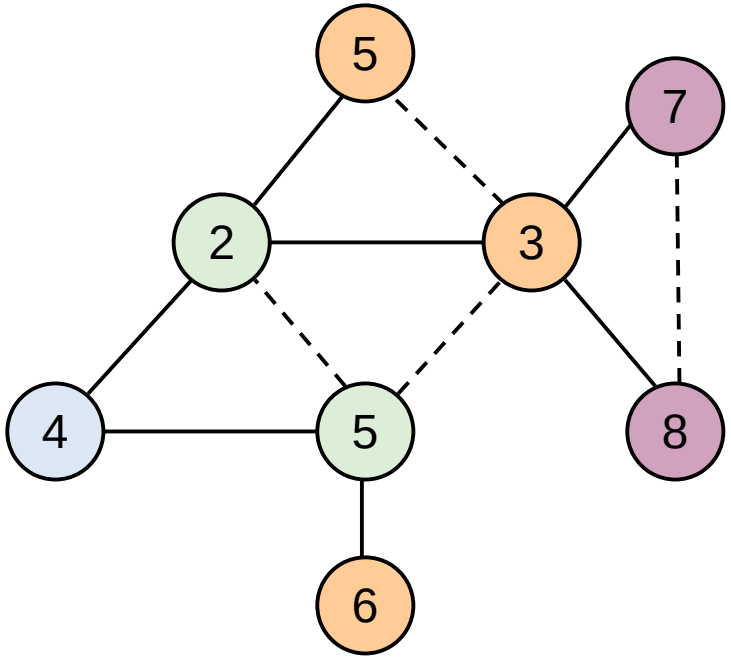
- Keep edge uv if v was first visited as a neighbor of u
- Why does BFS make a tree?
- What is true about the path from the starting vertex to any other vertex?



BFS Tree

We can use BFS to make a tree

- Keep edge uv if v was first visited as a neighbor of u
- Why does BFS make a tree?
- What is true about the path from the starting vertex to any other vertex?
- Is the BFS tree unique?



Next Time

- Depth-First search (DFS)
- Directed graphs
- Topological Sort